**1. Java & OOP (1–20)**

**1. What is Java?**

**Answer:**
Java is a **high-level, object-oriented, platform-independent programming language** developed by **James Gosling at Sun Microsystems in 1995 and now maintained by Oracle Corporation.

The main concept of Java is **"Write Once, Run Anywhere (WORA)"**, which means Java code compiled into **bytecode** can run on any system that has a **JVM (Java Virtual Machine)**.

Key features:

- Platform independent
- Object-oriented
- Secure
- Robust
- Multithreaded
- Distributed

Example:

```
class Hello{
  public static void main(String args[]){
    System.out.println("Hello Java");
  }
}
```

---

**2. What are the main features of Java?**

**Answer:**
Java provides several features that make it powerful for enterprise applications.

Important features:

1. **Platform Independent** – Java programs run on JVM.
2. **Object-Oriented** – Uses classes and objects.
3. **Secure** – No direct memory access.

4. **Multithreading** – Multiple tasks run simultaneously.
5. **Robust** – Automatic memory management with garbage collection.
6. **Portable** – Bytecode runs on different operating systems.

These features make Java widely used in **enterprise software, web applications, and Android development**.

---

**3. What is Object-Oriented Programming (OOP)?**

**Answer:**
OOP is a programming paradigm based on **objects and classes**, where objects represent real-world entities.

The main purpose of OOP is to **increase code reusability, maintainability, and scalability**.

The four pillars of OOP are:

1. Encapsulation
2. Inheritance
3. Polymorphism
4. Abstraction

Example:

```
class Car{
  String brand;
  void start(){
    System.out.println("Car started");
  }
}
```

---

**4. What is Encapsulation?**

**Answer:**
Encapsulation is the process of **wrapping data and methods together into a single unit (class)** and restricting direct access to variables.

This is achieved using:

- private variables
- public getter and setter methods

Advantages:

- Data hiding
- Better security
- Controlled access

Example:

```
class Student{
  private int age;

  public void setAge(int age){
    this.age = age;
  }

  public int getAge(){
    return age;
  }
}
```

---

**5. What is Inheritance?**

**Answer:**
Inheritance allows one class to **acquire properties and methods of another class**.

The class that inherits is called the **child class**, and the class being inherited is called the **parent class**.

Benefits:

- Code reuse
- Reduced redundancy
- Hierarchical classification

Example:

```
class Animal{
  void eat(){
    System.out.println("Animal eats");
  }
}

class Dog extends Animal{
  void bark(){
    System.out.println("Dog barks");
  }
}
```

---

**6. What is Polymorphism?**

**Answer:**
Polymorphism means **“many forms”**, where a single method behaves differently depending on the situation.

Types:

1. **Compile-time polymorphism** – Method overloading
2. **Runtime polymorphism** – Method overriding

Example (Overloading):

```
int add(int a,int b)
int add(int a,int b,int c)
```

Example (Overriding):

```
class Animal{
  void sound(){
    System.out.println("Animal sound");
  }
}

class Dog extends Animal{
  void sound(){
    System.out.println("Dog bark");
```

```
  }
}
```

---

**7. What is Abstraction?**

**Answer:**
Abstraction means **hiding internal implementation details and showing only functionality**.

It can be achieved using:

- Abstract classes
- Interfaces

Example:

```
abstract class Vehicle{
  abstract void start();
}
```

---

**8. Difference between Abstract Class and Interface?**

| Abstract Class | Interface |
|---|---|
| Can contain abstract and concrete methods | Mostly abstract methods |
| Supports constructors | No constructors |
| Uses extends | Uses implements |

---

**9. What is Method Overloading?**

**Answer:**
Method overloading occurs when multiple methods have the **same name but different parameters**.

Example:

```
void display(int a)
void display(String s)
```

---

**10. What is Method Overriding?**

**Answer:**
Method overriding occurs when a **child class provides its own implementation of a method defined in the parent class**.

Example:

```
class A{
  void show(){
    System.out.println("A class");
  }
}

class B extends A{
  void show(){
    System.out.println("B class");
  }
}
```

---

**11. What is JVM?**

**Answer:**
JVM (Java Virtual Machine) is the **runtime environment that executes Java bytecode**.

Responsibilities:

- Memory management
- Garbage collection
- Security
- Platform independence

---

**12. What is JDK?**

**Answer:**
JDK (Java Development Kit) is the **complete toolkit used for Java development**.

It contains:

- JRE
- Compiler (javac)
- Debugger
- Development tools

---

**13. What is JRE?**

**Answer:**
JRE (Java Runtime Environment) provides the **libraries and JVM required to run Java programs**.

JRE = JVM + Libraries

---

**14. What is the difference between == and .equals()?**

| == | equals() |
|---|---|
| Compares memory address | Compares object values |

---

**15. What is a Constructor?**

**Answer:**
A constructor is a **special method used to initialize objects**.

Example:

```
class Student{
  Student(){
    System.out.println("Constructor called");
  }
}
```

---

**16. What is Garbage Collection?**

**Answer:**
Garbage collection automatically **removes unused objects from memory**, preventing memory leaks.

---

**17. What is Multithreading?**

**Answer:**
Multithreading allows a program to **run multiple threads simultaneously**, improving performance.

---

**18. What is Exception Handling?**

**Answer:**
Exception handling is used to **handle runtime errors and maintain program flow**.

Keywords:

- try
- catch
- finally
- throw
- throws

---

**19. What is a Collection Framework?**

**Answer:**
Java Collections Framework provides **classes and interfaces to store and manipulate groups of objects**.

Examples:

- List
- Set
- Map

---

**20. Difference between ArrayList and LinkedList**

| ArrayList | LinkedList |
| --- | --- |
| Uses dynamic array | Uses doubly linked list |
| Faster random access | Faster insertion/deletion |

---

**2. Spring Boot (21–40)**

**21. What is Spring Boot?**

**Answer:**
Spring Boot is a framework that simplifies development of **Java-based web applications and microservices**.

Key features:

- Auto configuration
- Embedded server
- Minimal configuration

---

**22. What is @SpringBootApplication?**

Combination of:

@Configuration
@EnableAutoConfiguration
@ComponentScan

---

**23. What is Dependency Injection?**

Dependency Injection means **injecting required objects instead of creating them manually**.

Example:

@Autowired
UserService service;

---

**24. What is @RestController?**

Used to create REST APIs.

---

**25. What is @Autowired?**

Automatically injects required dependency.

---

**26. What is @Service?**

Marks a class as a **service layer component**.

---

**27. What is @Repository?**

Used for **database interaction layer**.

---

**28. What is @Controller?**

Handles **web requests in MVC applications**.

---

**29. What is Spring MVC?**

A web framework that follows **Model-View-Controller architecture**.

---

**30. What is Hibernate?**

Hibernate is an **ORM framework** used to map Java objects to database tables.

---

**31. What is JPA?**

JPA (Java Persistence API) is a **specification for ORM**.

---

**32. What is application.properties?**

Configuration file used to define:

- database connection

- server port
- logging

---

**33. What is an Embedded Server?**

Spring Boot includes servers like:

- Apache Tomcat
- Jetty

So we don't need to deploy WAR files.

---

**34. What is Spring Boot Starter?**

Preconfigured dependencies like:

- spring-boot-starter-web
- spring-boot-starter-data-jpa

---

**35. What is @GetMapping?**

Handles HTTP GET requests.

---

**36. What is @PostMapping?**

Handles HTTP POST requests.

---

**37. What is @RequestBody?**

Maps HTTP request body to Java object.

---

**38. What is @PathVariable?**

Extracts values from URL path.

---

**39. What is @RequestParam?**

Reads query parameters.

---

**40. What is Microservice Architecture?**

Microservices divide a large application into **small independent services**.

---

**3. REST API (41–50)**

**41. What is REST?**

REST is an architecture for building **web services using HTTP protocol**.

---

**42. What are HTTP methods?**

- GET
- POST
- PUT
- DELETE
- PATCH

---

**43. What is Stateless API?**

Each request contains **all required information**.

---

**44. What is JSON?**

JSON is a lightweight **data exchange format**.

---

**45. What is API endpoint?**

A URL where API resources are accessed.

Example:

/api/users

---

**46. What is HTTP Status Code?**

| Code | Meaning |
|---|---|
| 200 | Success |
| 201 | Created |
| 404 | Not Found |

---

**47. What is Idempotent method?**

Repeated requests produce **same result**.

Example: GET, PUT

---

**48. Difference between PUT and PATCH**

PUT → Update entire resource
PATCH → Update partial resource

---

**49. What is API security?**

Methods:

- JWT authentication
- OAuth
- API keys

---

**50. What is Postman?**

Postman is a tool used for **testing REST APIs**.

---

**4. SQL & Database (51–65)**

**51. What is SQL?**

SQL is used to **manage relational databases**.

---

**52. What is Primary Key?**

Unique identifier of table row.

---

**53. What is Foreign Key?**

Creates relationship between tables.

---

**54. What is JOIN?**

Combines rows from multiple tables.

Types:

- INNER
- LEFT
- RIGHT
- FULL

---

**55. What is Index?**

Improves database query performance.

---

**56. What is Normalization?**

Process of **reducing redundancy** in database.

---

**57. What is Denormalization?**

Combining tables to **improve read performance**.

---

**58. Difference between DELETE and TRUNCATE**

DELETE → removes rows
TRUNCATE → removes entire table data

---

**59. What is GROUP BY?**

Used for aggregation.

---

**60. What is HAVING?**

Used to filter aggregated results.

---

**5. Data Structures (66–80)**

**66. What is Data Structure?**

A method of organizing data efficiently.

---

**67. What is Array?**

Fixed-size data structure storing same type elements.

---

**68. What is Linked List?**

Nodes connected using pointers.

---

**69. What is Stack?**

LIFO structure.

Operations:

- push
- pop

---

**70. What is Queue?**

FIFO structure.

---

**71. What is Tree?**

Hierarchical data structure.

---

**72. What is Binary Tree?**

Each node has **maximum two children**.

---

**73. What is Graph?**

Collection of vertices and edges.

---

**74. What is HashMap?**

Stores **key-value pairs**.

---

**75. What is Time Complexity?**

Measures algorithm efficiency.

Example:
O(n), O(log n)

---

**6. Frontend (81–90)**

**81. What is HTML?**

Structure of webpage.

---

**82. What is CSS?**

Styles webpage.

---

**83. What is JavaScript?**

Adds interactivity to webpage.

---

**84. What is DOM?**

Document Object Model represents webpage structure.

---

**85. What is React?**

A JavaScript library developed by **Meta Platforms for building UI components.

---

**86. What is Angular?**

A frontend framework developed by **Google.

---

**87. Difference between React and Angular**

| React | Angular |
| --- | --- |
| Library | Framework |

---

**88. What is Responsive Design?**

Design that adapts to different screen sizes.

---

**89. What is Flexbox?**

CSS layout model.

---

**90. What is Bootstrap?**

Frontend CSS framework.

---

**7. Git & Version Control (91–100)**

**91. What is Git?**

Git is a **distributed version control system**.

---

**92. What is GitHub?**

A cloud platform for hosting Git repositories owned by **GitHub.

---

**93. What is Repository?**

Storage location for code.

---

**94. What is git clone?**

Copies repository to local system.

---

**95. What is git commit?**

Saves code changes.

---

**96. What is git push?**

Uploads code to remote repository.

---

**97. What is git pull?**

Fetch and merge remote changes.

---

**98. What is Branch?**

Separate development line.

---

**99. What is Merge?**

Combining branches.

---

**100. What is Pull Request?**

Request to merge code into main branch.